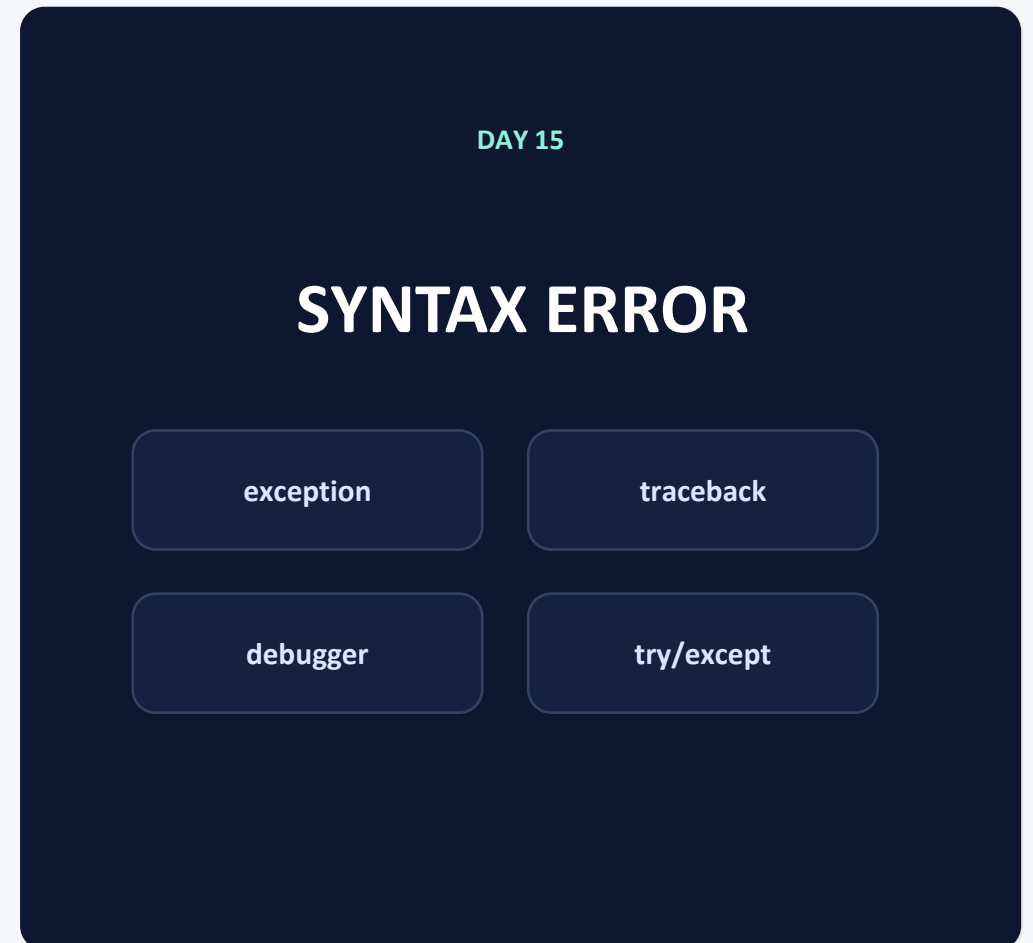


Ошибки и отладка

Читать сообщения об ошибках, различать синтаксические ошибки и исключения, использовать `traceback`, `try/except` и простую валидацию.



ЦЕЛЬ ДНЯ

Что ты сможешь после этого дня

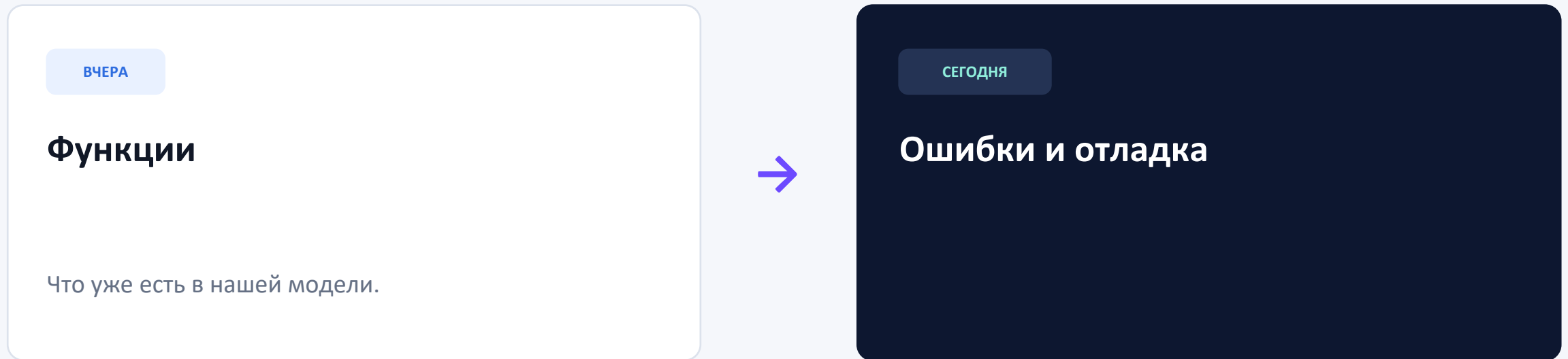
Конкретные действия вместо списка терминов.

- 01 Читать сообщения об ошибках, различать синтаксические ошибки и исключения, использовать `traceback`, `try/except` и простую валидацию.
- 02 Объяснить минимум 5 терминов темы своими словами.
- 03 Прочитать показанный пример кода и предсказать результат.
- 04 Самостоятельно решить похожую задачу после разбора.

СВЯЗЬ

Новая тема не появляется из ниоткуда

Связываем сегодняшний шаг с уже построенной картой курса.



Вчера: Функции. Сегодня мы добавляем следующий слой: ошибки и отладка.

ЗАЧЕМ

Зачем мне это?

Каждая тема решает конкретную проблему.

ПРОБЛЕМА → ПОЛЬЗА

Разработчик постоянно встречается ошибки. Навык спокойно прочитать тип, строку и причину экономит больше времени, чем попытка «угадывать» исправление.

Вопрос себе

Какую реальную проблему решает сегодняшняя тема?

Ответь до следующего слайда.

Сначала — человеческий образ

Потом обязательно связываем его с настоящей техникой.

БЫТОВОЙ ОБРАЗ

Ошибка — не авария, а сообщение от программы: traceback похож на маршрут до места, где произошло событие.

ПЕРЕВОД В IT

syntax error

ошибка в структуре кода, мешающая его разобрать

exception

объект ошибки, возникшей во время выполнения

traceback

цепочка вызовов, приведшая к исключению

Слой 1: **syntax error** • **exception** • **traceback**

Сначала смысл, затем профессиональный термин.

syntax error

ошибка в структуре кода, мешающая его разобрать

exception

объект ошибки, возникшей во время выполнения

traceback

цепочка вызовов, приведшая к исключению

Слой 2: debugger • try/except • raise

Сначала смысл, затем профессиональный термин.

**debugger**

инструмент пошагового выполнения и просмотра состояния

**try/except**

конструкция обработки ожидаемых исключений

**raise**

оператор явного создания исключения

Слой 3: validation • error handling

Сначала смысл, затем профессиональный термин.

validation

проверка входных данных до основной обработки

error handling

предусмотренная реакция программы на сбои

Не заучивай формулировку. Придумай свой пример использования каждого термина.

Слой 4: logging • finally

Сначала смысл, затем профессиональный термин.

**logging**

запись событий программы для анализа

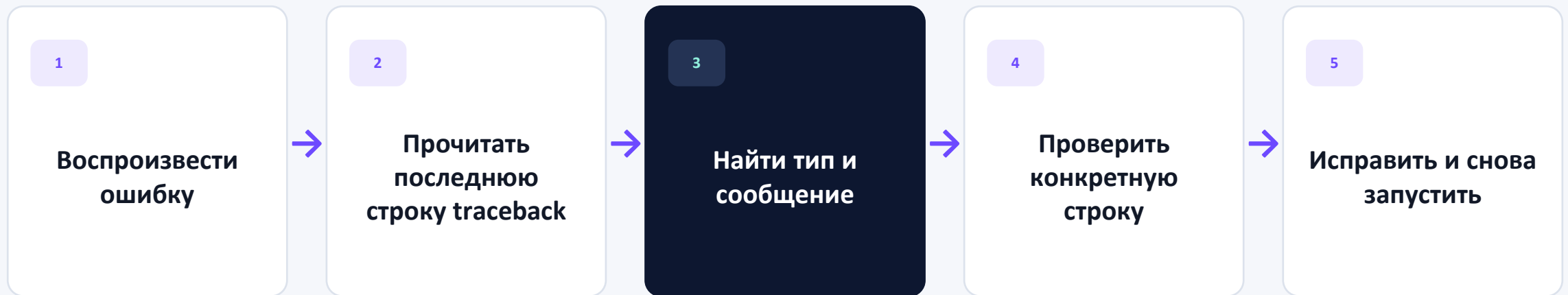
**finally**

блок, выполняемый независимо от наличия исключения

Не заучивай формулировку. Придумай свой пример использования каждого термина.

Как это работает по шагам

Проследи движение слева направо.



Проговори эту цепочку своими словами, не подсматривая в текст.

Первый минимальный пример

Смотрим только на одну основную идею.



day15_step1

```
try:  
    age = int("не число")  
except ValueError:  
    print("Возраст должен быть числом")
```

ЧТО ИСКАТЬ

- Где данные?
- Где действие?
- Где результат?
- Что изменится, если заменить одно значение?

Разбираем пример построчно

Код не должен оставаться магической конструкцией.

try:

Python выполняет эту строку сверху вниз.

age = int("не число")

Сохраняем значение или результат вычисления под понятным именем.

except ValueError:

Python выполняет эту строку сверху вниз.

print("Возраст должен быть числом")

Показываем результат, чтобы увидеть состояние программы.

РЕЗУЛЬТАТ

Код → выполнение → результат

Всегда связываем запись с тем, что реально получилось.



до запуска

```
try:  
    age = int("не число")  
except ValueError:  
    print("Возраст должен быть числом")
```

РЕЗУЛЬТАТ

Возраст должен быть числом

ИЗМЕНИЛИ

Теперь изменим пример

Сначала предскажи результат. Не запускай код сразу.

```
def parse_age(text):  
    try:  
        return int(text)  
    except ValueError:  
        return None  
  
print(parse_age("25"))  
print(parse_age("abc"))
```

changed



**Что появится в
результате?**

Сформулируй ответ словами.

ОТВЕТ

Сверяем прогноз

Не просто смотри ответ — объясни, почему он именно такой.

РЕЗУЛЬТАТ

25
None

ПОЧЕМУ

Python/система выполняет шаги в записанном порядке. Изменённые данные или действие дают новый результат.

Сравни с первым примером: что изменилось?

Как писать неправильно

Ошибка — отдельный учебный материал, а не повод её скрывать.

```
try:  
    result = 10 / 0  
except:  
    print("Ошибка")
```

wrong

СТОП

**Что здесь может сломаться
или ввести нас в
заблуждение?**

Сначала найди проблему сам.

Почему возникает проблема

Переводим техническую причину на обычный язык.

Голый эксерт ловит вообще всё и скрывает полезную информацию. Лучше ловить конкретное ожидаемое исключение.

Алгоритм отладки: симптом → конкретная строка/шаг → причина → минимальное исправление → повторная проверка.

Исправленный вариант

Меняем только то, что действительно было причиной.

```
try:  
    result = 10 / 0  
except ZeroDivisionError:  
    print("На ноль делить нельзя")
```

fixed

ПРАВИЛО

Не переписывай всё решение. Найди минимальное осмысленное исправление.

Что изменилось после исправления?

Ответь без запуска.

- 1 Исчезла ли исходная причина проблемы?
- 2 Какой результат/поведение теперь ожидаем?
- 3 Как проверить это одним маленьким примером?

Сначала увидим полностью разобранный пример

Только после него будет самостоятельная практика.

ЗАДАЧА

Безопасно преобразовать возраст: вернуть число для корректного ввода и понятное сообщение для некорректного.

Не начинаем с кода. Сначала переводим задачу на человеческий язык.

КАК ДУМАТЬ

Как рассуждает программист

Разбиваем большую формулировку на маленькие действия.

- 1 **Функция получает текст.**
- 2 Пробуем `int(text)`.
- 3 Если всё хорошо — возвращаем число.
- 4 Если `ValueError` — возвращаем `None`.
- 5 Снаружи решаем, что показать пользователю.

Шаг 1: собираем основу

Не показываем 20 строк сразу — строим решение частями.

```
def parse_age(text):  
    try:  
        return int(text)  
    except ValueError:  
        return None  
  
age = parse_age("oops")
```

solution_part1

Проверь

- Какие данные уже есть?
- Какой шаг плана реализован?
- Что ещё осталось добавить?

ИТОГОВЫЙ КОД

Шаг 2: итоговое решение

Теперь видим программу/запрос целиком.



```
def parse_age(text):  
    try:  
        return int(text)  
    except ValueError:  
        return None  
  
age = parse_age("oops")  
if age is None:  
    print("Введите возраст цифрами")  
else:  
    print(age)
```

solution_final

Проверяем результат

Сравниваем то, что ожидали, с тем, что получили.

РЕЗУЛЬТАТ

Введите возраст цифрами

ГЛАВНОЕ

Задача была решена не потому, что мы «знали готовый код», а потому что сначала построили понятный план.

ПОПРОБУЙ САМ

Практика 1

Задача очень похожа на разобранную — но теперь первый ход за тобой.

Напиши `parse_quantity(text)`, которая возвращает `int` или `None`.

Сначала выпиши 2–5 шагов решения словами.

ПОДСКАЗКА

Подсказка — только направление

Попробуй ещё раз до просмотра готового решения.

Это почти тот же шаблон, что `parse_age: try → int → except ValueError`.

Вопрос себе: какие конструкции сегодняшнего дня нужны, а какие пока лишние?

Один из правильных вариантов

Сравни логику, а не количество строк.



```
def parse_quantity(text):  
    try:  
        return int(text)  
    except ValueError:  
        return None  
  
print(parse_quantity("7"))  
print(parse_quantity("seven"))
```

practice_solution

Ожидаемый результат

Если твой код устроен иначе, но даёт правильное и понятное решение — это нормально.

РЕЗУЛЬТАТ

7
None

Сравни

- Результат
- Понятность имён
- Лишние шаги
- Обработку ошибок

Практика 2 — мини-проект

Соединяем несколько элементов дня в одну маленькую закончённую задачу.

ЗАДАНИЕ

Добавь валидацию: количество должно быть не только числом, но и больше нуля.

Попробуй сначала без подсказки. Разрешено подсматривать в сегодняшние примеры.

Разбор мини-проекта

Вот один рабочий путь.



```
def parse_positive(text):
    try:
        value = int(text)
    except ValueError:
        return None
    if value <= 0:
        return None
    return value

print(parse_positive("3"))
print(parse_positive("-1"))
```

mini_project

РЕЗУЛЬТАТ ПРОЕКТА

Что должно получиться

Связываем итог с задачей и проверяем смысл.

РЕЗУЛЬТАТ

3
None

Объясни каждую важную строку/команду своими словами. Если не можешь — вернись к конкретному месту, а не перечитывай весь день.

Четыре ловушки новичка

Лучше увидеть их здесь, чем случайно закрепить как привычку.

01 Игнорировать текст `traceback`.

02 Ловить `Exception` без причины.

03 Использовать `try/except` вместо обычной логической проверки.

04 Молча подавлять ошибку и продолжать с некорректными данными.

Проверь себя: 1–2

Не листай дальше, пока не сформулируешь ответ.

- 1 Что такое «syntax error» и какую проблему это понятие помогает описать?
- 2 Чем «syntax error» отличается от «debugger»? Объясни без копирования определения.

Проверь себя: 3–4

Здесь проверяем понимание кода и процесса мышления.

- 3 Какой результат даёт первый минимальный пример и почему?
- 4 Какую ошибку/риск мы разобрали сегодня и какое минимальное исправление нужно?

ОТВЕТЫ

Ответы к вопросам 1–2

Формулировка может отличаться — важен смысл.

1 **syntax error — ошибка в структуре кода, мешающая его разобрать.**

2 **syntax error: ошибка в структуре кода, мешающая его разобрать. debugger: инструмент пошагового выполнения и просмотра состояния.**

ОТВЕТЫ

Ответы к вопросам 3–4

Сверяем причинно-следственную связь.

3 Возраст должен быть числом

4 Голый эксперт ловит вообще всё и скрывает полезную информацию. Лучше ловить конкретное ожидаемое исключение.

ИТОГ

Сегодня мы научились

Не перечисляем слова — фиксируем новые способности.

- ✓ Читать сообщения об ошибках, различать синтаксические ошибки и исключения, использовать `traceback`, `try/except` и простую валидацию.
- ✓ Читать и объяснять пример темы «Ошибки и отладка».
- ✓ Находить типичную ошибку и формулировать причину.
- ✓ Решать похожую задачу по шагам, а не угадывать готовый код.

ДАЛЬШЕ

Что будет завтра

Связываем сегодняшний результат со следующим шагом.

Модули, пакеты и зависимости

Завтра: Модули, пакеты и зависимости. Сегодняшняя модель станет опорой для следующего шага.

Сегодняшняя тема не заканчивается здесь — она станет инструментом в следующем дне.